Extended Abstract

Motivation The objective of our extension is to improve the mathematical reasoning skills of the Qwen 2.5 0.5B Base model by allowing the model to query a simple arithmetic calculator tool during training with the SFT and RLOO loss objectives. We wish to apply this to the downstream task, Countdown, which gives the model a list of numbers and a target number and asks it to show its reasoning in obtaining the target number using arithmetic operations on the given list. Large language models are notoriously bad at math Yuan et al. (2023) due to replicating patterns rather than actually performing the correct mathematical operations. Thus, allowing the LLM to use a calculator during training should lead it to provide more correct answers during inference.

Method For the SFT extension, we followed an approach inspired by Schick et al. (2023), augmenting a dataset with prompts that simulate calling a calculator API. The model was finetuned on this augmented data to learn when and how to invoke an external calculator, following a two-step training process similar to Toolformer. Although we encountered infrastructure issues that limited full training of the extended model, we implemented the core logic and generated the first iteration of the model with the augmented data. We did not get to the RLOO extension at all.

Implementation In terms of implementation, we only implemented and trained the base SFT and RLOO models due to time constraints, spot capacity issues, and a host of other issues. However, We developed complete data generation pipelines for SFT, RLOO, and SFT + extension. Furthermore, RLOO was not trained for as long as it needed to be due to spot capacity issues.

Results We found that the RLOO and SFT models both surpassed the baselines for their respective leaderboards, with scores of 0.352 and 0.329 respectively.

Discussion We predict that RLOO would perform better with a full epoch of training.

Conclusion We did not complete everything we wanted to achieve with this project. For next steps, we would finish RLOO training, implement the extension for it, and complete the extension for SFT.

LLMs Are Bad At Math: Improving Math Reasoning with RL and External Tooling

Sydney Faux
Department of Symbolic Systems
Stanford University
sfaux@stanford.edu

Mar Bigolin Groff
Department of Computer Science
Stanford University
mbgroff@stanford.edu

Abstract

The objective of our extension is to improve the mathematical reasoning skills of the Qwen 2.5 0.5B Base model by allowing the model to query a simple arithmetic calculator tool during training with the SFT and RLOO loss objectives. We wish to apply this to the downstream task, Countdown, which gives the model a list of numbers and a target number and asks it to show its reasoning in obtaining the target number using arithmetic operations on the given list. Unfortunately, we were not able to implement and train the extension fully due to several factors. However, we were able to hit the baselines for regular SFT and RLOO training, achieving 0.352 win rate on our own generated leaderboard evaluation and 0.328 on the actual leaderboard evaluation respectively.

1 Introduction

Large language models are notoriously bad at math Yuan et al. (2023) due to replicating sometimes false patterns rather than actually performing the correct mathematical operations. External tooling has emerged Schick et al. (2023) as a way to improve the accuracy of LLM outputs, including on mathematical reasoning tasks by training the LLM to query relevant external tools. Furthermore, reinforcement learning paired with external tooling Jin et al. (2025) has also been shown to improve accuracy of LLM outputs.

Our aim is to improve the mathematical reasoning skills of the Qwen 2.5 0.5B Base model for the Countdown task by allowing the model to query a simple arithmetic calculator during training with the SFT and RLOO objectives.

Thus, our research question is: Can we improve the mathematical reasoning skills of an LLM by allowing it to call an external calculator tool during training?

2 Related Work

Retrieval-Augmented Generation (RAG)

In their paper, *Retrieval-augmented generation for large language models*, Gao et al. (2024) allow LLMs to access external information for response generation by retrieving relevant information from a data store using the input prompt and subsequently concatenating it to the input to be fed into the model. However, this method falls short of our aim for our extension due to the lack of control over what information is being retrieved as discussed in the paper.

External Tools with SFT

In their paper, *Toolformer: Language Models Can Teach Themselves to Use Tools*, Schick et al. (2023) train a model using supervised fine-tuning (SFT) to know when to query external tools, including

a calculator, for response generation. They do so by letting the model self-annotate the dataset for potential API calls and then train the model with inputs that are interleaved with the results of the API calls. Thus, this paper offers a solid foundation for implementing our extension with the SFT objective.

External Tools with RL

In their paper, *Search-R1: Training LLMs to Reason and Leverage Search Engines with Reinforcement Learning*, Jin et al. (2025) use search engine tools with reinforcement techniques to improve their reasoning capabilities. They do so by using multi-turn search engine calls to produce a LLM response rollout interleaved with the results of the queries that is used to calculate the Proximal Policy Optimization (PPO) and Group Relative Policy Optimization (GRPO) loss objectives. They found that their method results in improvements of 41% and 20% for two LLMS over RAG baselines. Thus, their paper provides a solid foundation for our work as it combines RL techniques with external tooling.

3 Methods for SFT Extension

Data Augmentation

For the SFT extension, approach is heavily based on the work in Schick et al. (2023). Similar to Toolformer, we augmented the initial dataset to include data with prompt that call the calculator tool (Table 1) and outputs for how the model should call the tool, using [Calculate (expression) -> result] (Table 2).

Original Prompt	Augmented Prompt
A conversation between User and Assistant. The user asks a question, and the Assistant solves it. The assistant first thinks about the reasoning process in the mind and then provides the user with the answer. User: Using the numbers [46, 27, 86], create an equation that equals 13. You can use basic arithmetic operations (+, -, *, /) and each number can only be used once. Show your work in <think> </think> tags. And return the final answer in <answer> </answer> tags, for example <answer> (1 + 2) / 3 </answer>. Assistant: Let me solve this step by step.	A conversation between User and Assistant. The user asks a question, and the Assistant solves it. The assistant first thinks about the reasoning process in the mind and then provides the user with the answer. User: Using the numbers [46, 27, 86], create an equation that equals 13. You can use basic arithmetic operations (+, -, *, /) and each number can only be used once. Show your work in <think> </think> tags. If you need to perform an arithmetic operation, you can call the Calculator API by writing '[Calculator(expression) -> result]' where 'expression' is the expression to be computed and 'result' is the result from the API, for example [Compute(2+1) -> 3]. And return the final answer in <answer> </answer> tags, for example <answer> (1 + 2) / 3 </answer>. Assistant: Let me solve this step by step.

Table 1: Original Prompt for Countdown Dataset vs. Augmented Prompt

Original Completion	Augmented Completion
<think> Let me analyze this step by step: 1. First, let's look for numbers close to 69 or factors/multiples of 69: - 69 = 3 x 23 - We have 39 which is close to 69 (need +30) - 91 is also relatively close to 69 (need -22) 2. Let's try working with 91 since it's close to 69: - 91 - 22 = 69 - Can we get 22 somehow? - 39 - 2 = 37, too large - 83 - 39 = 44, too large 3. Let's try working with 39: - Need +30 to get to 69 - 83 - 39 = 44 - 91 - 83 = 8 - 2 x 39 = 78, too large 4. New approach: - 83 - 39 = 44 - 91 / 2 = 45.5 (can't use this as we need integers) - What if we subtract 83 from 91? - 91 - 83 = 8 - Then multiply 8 by 2 = 16 - Add 39 = 55 - This isn't helping 5. Let's try one more approach: - 39 x 2 = 78 - 91 - 83 = 8 - 78 - 8 = 70 - That's very close! Just need -1 - Working backwards: ((39 * 2) - (91 - 83)) = 70 - Actually: ((39 * 2) - (91 - 83)) - 1 = 69 The sequence works! </think> <answer>(39 * 2) - (91 - 83)</answer>"	<think> Let me analyze this step by step: 1. First, let's look for numbers close to 69 or factors/multiples of 69: - 69 = [Compute(3 * 23) -> 69.0] - We have 39 which is close to 69 (need +30) - 91 is also relatively close to 69 (need -22) 2. Let's try working with 91 since it's close to 69: - [Compute(91 - 22) -> 69.0] - Can we get 22 somehow? - [Compute(39 - 2) -> 37.0], too large - [Compute(83 - 39) -> 44.0], too large 3. Let's try working with 39: - Need +30 to get to 69 - [Compute(83 - 39) -> 44.0] - [Compute(91 - 83) -> 8.0] - [Compute(2 * 39) -> 78.0], too large 4. New approach: - [Compute(83 - 39) -> 44.0] - [Compute(91 / 2) -> 45.5] (can't use this as we need integers) - What if we subtract 83 from 91? - [Compute(91 - 83) -> 8.0] - Then multiply 8 by 2 = 16 - Add 39 = 55 - This isn't helping 5. Let's try one more approach: - [Compute(39 * 2) -> 78.0] - [Compute(91 - 83) -> 8.0] - [Compute(78 - 8) -> 70.0] - That's very close! Just need -1 - Working backwards: [Compute((39 * 2) - (91 - 83)) -> 70.0] - Actually: [Compute(((39 * 2) - (91 - 83)) - 1) -> 69.0] The sequence works! </think> <answer>(39 * 2) - (91 - 83)</answer>"

Table 2: Original Competion for Countdown Dataset vs. Augmented Completion

Algorithm

To get the model to call the external calculator during training, we will use a similar algorithm found in Schick et al. (2023):

1. Use the augmented data to finetune the base model with SFT objective.
2. Use finetuned model to generate outputs with calculator tags.
3. Use generated dataset to retrain the model.
4. Run evaluation with augmented prompts from `countdown_heldout_prompts.json` by input it to customized model generate function that calls the calculator whenever Calculator tags show up in the completion.

4 Experimental Setup

Datasets

For training our base SFT model and the SFT extension, we used the warmstart dataset, `"Asap7772/cog_behav_all_strategies"`. For both data pipelines, we formatted each example into a prompt-completion structure following the chat format expected by the Qwen2.5 tokenizer. The pipeline tokenizes both user queries and assistant responses together, then masks the user tokens in the label field by replacing them with -100 to prevent them from contributing to the loss during training. The resulting tokenized dataset, including input IDs, attention masks, and labels, is converted into PyTorch tensors and saved to disk for later use in model training. For the extension, we randomly split into 90% training and 10% test sets using a fixed seed.

For finetuning with the RLOO objective, we used the official Countdown dataset, `"Countdown-Task-3to4"`. For the data pipeline, we constructed the input prompts out of the numbers and target columns in the original dataset to use for sampling generation in the RLOO loss objective calculation.

Hyperparameters and Training Strategy

Unfortunately, we were not able to complete the training for the SFT model with calculator extension because of spot capacity issues on AWS and we did not even implement the extension of RLOO due to time constraints, so we just have results for regular SFT and RLOO.

For the base SFT model, we trained it with 2 epochs, a learning rate of 3.4e-05, and batch size of 2. For the RLOO model, we only trained for 10 batches in the first epoch (early partial result from initial testing of RLOO objective), a learning rate of 1e-06, batch size of 4, and a k of 2. We tried to run for a full epoch later with a testing loop that calculated average scores instead of loss, but failed do to AWS spot capacity issues. For RLOO training, we also used a KL divergence proxy to prevent the model from straying to far from the base SFT model.

For the reward function for RLOO training, we changed the the original scores so that an output with no found equation had a -1.0 reward, an output with a wrong equation had a 0.5 reward, and output that was totally correct has a reward of 1.0.

All models were trained on a g6e.xlarge instance and with the AdamW optimizer.

Evaluation

We evaluated the base SFT model on the held out prompts of the first leaderboard. Our evaluation score was calculated internally because the first leaderboard had been taken down by that point. We evaluated the RLOO model on the held out prompts of the 2nd leaderboard.

5 Results

5.1 Quantitative Evaluation

Table 3: Win Rates and Losses for SFT and RLOO

Method	Win Rate	Train Loss	Test Loss
SFT	0.352	0.18	0.231
RLOO	0.329	-3.92	-10.6

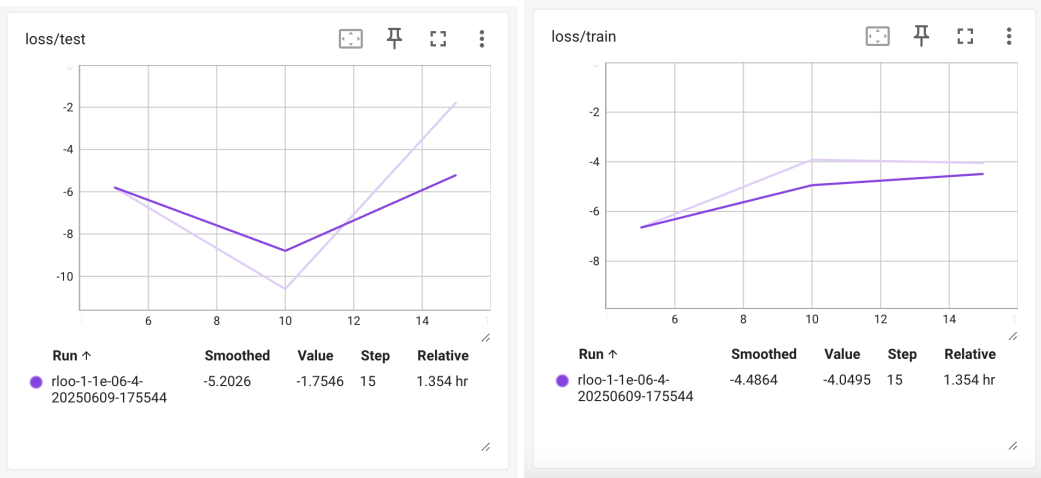

Figure 1: Train and Test Plots for RLOO

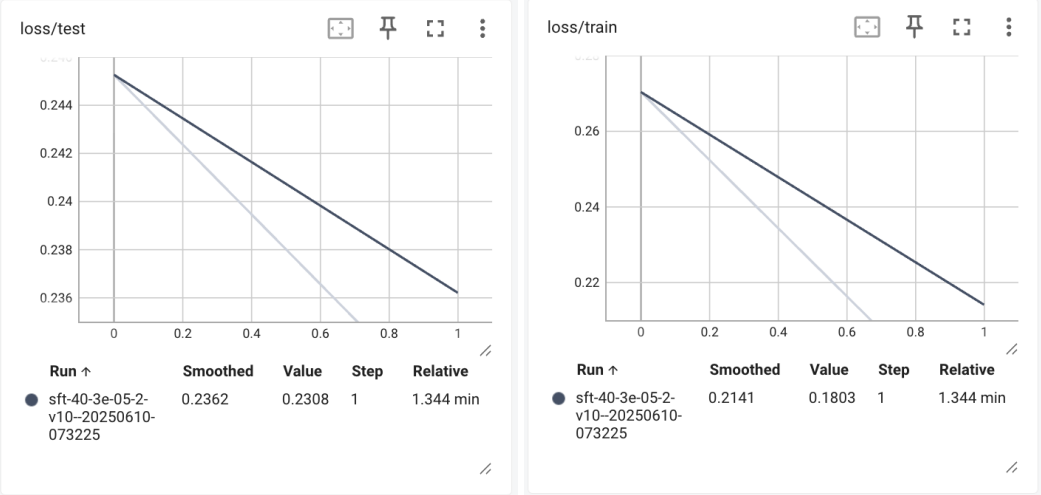

Figure 2: Train and Test Plots for SFT Base

6 Discussion

Our experiments show the promise and challenges of extending SFT models with tool-use capabilities. Since initial completion generations contained arithmetic errors, the integration of an external tool like a calculator showed promise in avoiding that wrong results were evaluated due to lack of mathematical training. Although we were unable to fully train and evaluate the SFT+calculator model due to infrastructure limitations (e.g., AWS spot instance interruptions), our pipeline design and partial results indicate that this direction is viable with more stable compute access and time.

For the RLOO model, even though the win rate is lower than the SFT Model (Table 3), these are positive results considering the model was not trained with the full dataset and there being more held out prompts for RL evaluation than SFT. We predict that if RLOO was allowed to run for the full epoch that it would outperform base SFT from initial recorded losses and scores from an attempted full epoch run. Furthermore, as we can see, the losses for the RLOO model started negative which is a good sign because it means that the model is producing outputs that are optimizing the reward function. (Figure 1)

Future work should explore scaling this approach to more complex integrations of mathematical tools, particularly in RLOO, as well as a more rigorous comparison between supervised and reinforcement objectives in tool-augmented models.

7 Conclusion

Use of an external calculator shows great promise in improving LLMs results in processing mathematical equations. Even though we were unable to run the final versions of RLOO and SFT with the calculator tool, the lack of ability of the model to properly evaluate equations is a sign that LLM models should have access to external tools.

8 Team Contributions

- **Sydney Faux:** She helped implement and train the base SFT model and its data generation pipeline. She also implemented and trained the RLOO model.
- **Mar Bigolin Groff:** They helped implement and train the base SFT model. They implemented the data generation pipeline for SFT with extension and attempted to train it.

Changes from Proposal We did not end up fully implementing the extension for SFT and RLOO.

References

Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang, and Haofen Wang. 2024. Retrieval-Augmented Generation for Large Language Models: A Survey. arXiv:2312.10997 [cs.CL] https://arxiv.org/abs/2312.10997

Bowen Jin, Hansi Zeng, Zhenrui Yue, Jinsung Yoon, Sercan Arik, Dong Wang, Hamed Zamani, and Jiawei Han. 2025. Search-R1: Training LLMs to Reason and Leverage Search Engines with Reinforcement Learning. arXiv:2503.09516 [cs.CL] https://arxiv.org/abs/2503.09516

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. arXiv:2302.04761 [cs.CL] https://arxiv.org/abs/2302.04761

Zheng Yuan, Hongyi Yuan, Chuanqi Tan, Wei Wang, and Songfang Huang. 2023. How well do Large Language Models perform in Arithmetic tasks? arXiv:2304.02015 [cs.CL] https://arxiv.org/abs/2304.02015